

Understanding Redux

Maintaining and controlling state in an app is something that quickly becomes complicated as soon as our app is larger than a Todo app, especially if we have multiple views, models, and dependencies between them. The situation is complicated by multiple types of states, such as cached data, server responses, and data that only lives locally when you work with the app. Changing the state is made more complicated because multiple actors, synchronous, and asynchronous code can change the state. The end result with a growing app is, sooner or later, a non-deterministic system. The problem with such a system is that you lose predictability, which in turn means you may have bugs that are hard to produce, and it makes the app and its data hard to reason with. We crave order and predictability, but we get neither.

To try to address the problem, we covered the Flux pattern in a previous part. All is well and good, right? We don't need another pattern. Or do we? Well, Flux has problems. One of the problems is that your data is divided up into several stores. Why is that a problem, you wonder? Imagine you have an action that triggers in multiple stores. It's easy to forget to handle an action in all stores. So, that issue is more of a management problem. Another problem with multiple stores is that it's hard to get a good overview of what your state consists of. Updates are another problem we have with Flux. Sometimes you have a lot of updates; updating the state and the order matters. In Flux, this is handled with a construct called `waitFor`. The idea is that you should be able to specify what should happen in which order. That's all well and good, but imagine that this is spread out over many modules; it becomes hard to keep track of and is thereby error prone.

Mutation and asynchronous behavior are two concepts that are hard to deal with. Mutation means we change the data. Asynchronous means something takes time to complete; when it does it may mutate a state. Imagine mixing synchronous and asynchronous operations that all update state. We realize it's not easy to keep track of the code because of this, and mixing that in with state mutation makes it all the more complicated.

This leads us to what Redux can do for us, which is to make our mutations predictable, but it also gives us one store, one single source of truth.

In this part, you will learn:

- The core concepts
- How data flows
- How to put your skills into practice by building your own mini implementation of Redux
- How to deal with AJAX in the context of Redux
- Some sound best practices

Principles

Redux rests on three principles:

- Single source of truth: We have one place where all our data lives.
- State is read-only: No mutation; there is only one way to change state and that is through an action.
- Changes are made with pure functions: A new state is produced by taking the old state, applying the change, and producing the new state; the old state is never changed.

Let's explore these bullet points one by one.

Single source of truth

The data lives in a single store in Redux and not a multiple store like in Flux. The data is represented by one object tree. This brings about a lot of benefits, such as:

- It is easier to see what your application knows at any given point, so it is easy to serialize or deserialize it.
- It is easier to work with in development, and easier to debug and inspect.
- It is easier to do things such as undo/redo if all applied actions produce a new state.

An example of a single store can look like the following:

```
// principles/store.js

class Store {
  getState() {
    return {
      jedis: [
        { name: "Yoda", id: 1 },
        { name: "Palpatine", id: 2 },
        { name: "Darth Vader", id: 3 }
      ],
      selectedJedi: {
        name: "Yoda",
        id: 1
      }
    };
  }
}

const store = new Store();
console.log(store.getState());

/*
{
  jedis: [
    { name: 'Yoda', id: 1 },
    { name: 'Palpatine', id: 2 },
    { name: 'Darth Vader', id: 3 }
  ],
  selectedJedi: {
    name: 'Yoda', id: 1
  }
}
*/
```

As you can see, this is just an object.

Read-only states

We want to ensure we only have one way to alter state, and that is through mediators called actions. An action should describe the intent of the action as well as the data that should be applied to the current state. An action is dispatched by us using a `store.dispatch(action)`. The action itself should look like the following:

```
// principles/action.js

// the action
let action = {
  // expresses intent, loading jedis
  type: "LOAD_JEDIS",
  payload:[
    { name: "Yoda", id: 1 },
    { name: "Palpatine", id: 2 },
    { name: "Darth Vader", id: 3 }
  ]
};
```

At this point, let's try to implement what a store might actually look like and what it initially contains:

```
// principles/storeII.js

class Store {
  constructor() {
    this.state = {
      jedis: [],
      selectedJedi: null
    }
  }
  getState() {
    return this.state;
  }
}

const store = new Store();

console.log(store.getState());
// state should now be
/*
{
  jedis : [],
  selectedJedi: null
}
*/
```

We can see that it is an object that consists of two properties, `jedis`, which is an array, and `selectedJedi`, which is an object holding an object that we select. At this point, we want to dispatch an action, which means we will take our old state, as shown in the preceding code, and produce a new state. The action we described earlier should change the `jedis` array and replace the empty array with the incoming array. Remember, though, we don't mutate the existing store object; we simply take it, apply our change, and produce a new object. Let's dispatch our action and see the end result:

```
// principles/storeII-with-dispatch.js

class Store {
  constructor() {
    this.state = {
      jedis: [],
      selectedJedi: null
    }
  }

  getState() {
    return this.state;
  }

  dispatch(action) {
    // to be implemented in later sections
  }
}

// the action
let action = {
  type: 'LOAD_JEDIS',
  payload:[
    { name: 'Yoda', id: 1 },
    { name: 'Palpatine', id: 2 },
    { name: 'Darth Vader', id: 3 }
  ]
}

// dispatching the action, producing a new state
store.dispatch(action);

console.log(store.getState());
// state should now be
/*
{
  jedis : [
    { name: 'Yoda', id: 1 },
```

```

    { name: 'Palpatine', id: 2 },
    { name: 'Darth Vader', id: 3 }
  ],
  selectedJedi: null
}
*/

```

The preceding code is pseudo code as it doesn't actually produce the intended result, yet. We will learn to implement a store in later parts. OK, so now our state has changed and our incoming array has replaced the empty array we used to have. We again repeat that we have not mutated the existing state, but instead produced a new state given the old state and our action. Let's look at the next section on *pure functions* and further explain what we mean.

Changing states with pure functions

In the previous section, we introduced the concept of the action and how that was the mediator through which we were allowed to change our state. We didn't change the state, though, in the normal sense of the word, but rather took the old state, applied the action, and produced a new state. To accomplish this, we need to use a pure function. In the context of Redux those are called reducers. Let's write ourselves a reducer:

```

// principles/first-reducer.js

module.exports = function reducer(state = {}, action) {
  switch(action.type) {
    case "SELECT_JEDI":
      return Object.assign({}, action.payload);
    default:
      return state;
  }
}

```

We highlight the pure aspect of the preceding reducer. It takes our `selectedJedi` from the `action.payload`, it copies it using the `Object.assign()`, assigns it, and returns the new state.

What we have written is a `reducer` that switches, depending on the action we try to perform, and carries out the change. Let's put this pure function into use:

```
const reducer = require("./first-reducer");

let initialState = {};
let action = { type: "SELECT_JEDI", payload: { id: 1, name: "Jedi" } };
let state = reducer(initialState, action);
console.log(state);

/* this produces the following:
{ id: 1, name: 'Yoda' }
*/
```

Core concepts

In React, we are dealing with three core concepts, which we have already introduced the state, the action, and the reducer. Now, let's dive in deeper and really get a sense of how they fit together and how they work.

Immutability patterns

The whole point of the state is to take an existing state, apply an action to it, and produce a new state. It can be written like this:

```
old state + action = new state
```

Imagine, if you were performing basic calculations, then you would start writing it like this:

```
// sum is 0
let sum = 0;

// sum is now 3
sum +=3;
```

The Redux way of doing things, though, is to change the preceding to:

```
let sum = 0;
let sumWith3 = sum + 3;
let sumWith6 = sumWith3 + 3;
```

We don't mutate anything but rather produce a new state for everything we do. Let's look at different constructs and what it means in practice to not mutate.

Changing a list

There are two operations we can perform on a list:

- Add item(s) to a list
- Remove item(s) from a list

Let's take the first bullet point and make this change in the old way and then make the Redux way:

```
// core-concepts/list.js

// old way
let list = [1, 2, 3];
list.push(4);

// redux way
let immutablelist = [1, 2, 3];
let newList = [...immutablelist, 4];

console.log("new list", newList);
/*
  [1, 2, 3, 4]
*/
```

The preceding code takes the old list and its items and creates a new list containing the old list plus our new member.

For our next bullet, to remove an item, we do this:

```
// core-concepts/list-remove.js

// old way
let list = [1, 2, 3];
let index = list.indexOf(1);
list.splice(index, 1);

// redux way
let immutableList = [1, 2, 3];
let newList = immutableList.filter(item => item !== 1);
```

As you can see, we produce a list not containing our item.

Changing an object

Changing an object is about changing properties on it as well as adding properties to it. First off, let's look at how to change existing values:

```
// core-concepts/object.js

// the old way
let anakin = { name: "anakin" };
anakin.name = "darth";
console.log(anakin);

// the Redux way
let anakinRedux = { name: "anakin" };
let darth = Object.assign({}, anakinRedux, { name: "darth" });

console.log(anakinRedux);
console.log(darth);
```

That covers the existing case. What about adding a new property? We can do that like so:

```
// core-concepts/object-add.js

// the old way
let anakin = { name: "anakin" };
console.log("anakin", anakin);

anakin["age"] = "17";
console.log("anakin with age", anakin);

// the Redux way
let anakinImmutable = { name: "anakin" };
let anakinImmutableWithAge = Object.assign({}, anakinImmutable, { age: 17
});

console.log("anakin redux", anakinImmutable);
console.log("anakin redux with age", anakinImmutableWithAge);
```

Using reducers

In the previous section, we covered how to change state in the old way and how to do it in the new Redux-like way. The reducers are nothing more than pure functions; pure in the sense that they don't mutate but produce a new state. A reducer needs an action to work though. Let's deepen our knowledge on Reducers and Actions. Let's create an action meant to add things to a list and a reducer that goes with it:

```
// core-concepts/jedilist-reducer.js

let actionLuke = { type: "ADD_ITEM", payload: { name: "Luke" } };
let actionVader = { type: "ADD_ITEM", payload: "Vader" };

function jediListReducer(state = [], action) {
  switch(action.type) {
    case "ADD_ITEM":
      return [... state, action.payload];
    default:
      return state;
  }
}

let state = jediListReducer([], actionLuke);
console.log(state);
/*
[ { name: 'Luke' } ]
*/

state = jediListReducer(state, actionVader);
console.log(state);
/*
[ { name: 'Luke' }, { name: 'Vader' } ]
*/

module.exports = jediListReducer;
```

OK, so now we know how to deal with lists; what about objects? We again need to define an action and a reducer:

```
// core-concepts/selectjedi-reducer.js

let actionPerson = { type: "SELECT_JEDI", payload: { id: 1, name: "Luke" } };
let actionVader = { type: "SELECT_JEDI", payload: { id: 2, name: "Vader" } };

function selectJediReducer({}, action) {
```

```

    switch (action.type) {
      case "SELECT_JEDI":
        return Object.assign({}, action.payload);
      default:
        return state;
    }
  }

  state = selectJediReducer({}, actionPerson);
  console.log(state);
  /*
  { name: 'Luke' }
  */

  state = selectJediReducer(state, actionVader);
  console.log(state);
  /*
  { name: 'Vader' }
  */

  module.exports = selectJediReducer;

```

What we see here is how one object completely replaces the content of another object by invoking `SELECT_JEDI`. We also see how we use `Object.assign()` to ensure we only copy over the values from the incoming object.

Merging all reducers together

OK, so we now have a reducer for handling our list of `jedis` as well as a reducer dedicated to handling selections of a specific `jedi`. We mentioned before that, in Redux, we have a single store where all our data lives. Now it's time to create that single store. This can be easily accomplished by creating the following function `store()`:

```

// core-concepts/merged-reducers.js

function store(state = { jedis: [], selectedJedi: null }, action) {
  return {
    jedis: jediListReducer(state.jedis, action),
    selectedJedi: selectJediReducer(state.selectedJedi, action)
  };
}

let newJediActionYoda = { type: "ADD_ITEM", payload: { name: "Yoda" } };
let newJediActionVader = { type: "ADD_ITEM", payload: { name: "Vader" } };
let newJediSelection = { type: "SELECT_JEDI", payload: { name: "Yoda" } };

```

```

let initialState = { jedis: [], selectedJedi: {} };

let state = store(initialState, newJediActionYoda);
console.log("Merged reducers", state);
/*
  {
    jedis: [{ name: 'Yoda' }],
    selectedJedi: {}
  }
*/

state = store(state, newJediActionVader);
console.log("Merged reducers", state);
/*
  {
    jedis: [{ name: 'Yoda' }, {name: 'Vader'}],
    selectedJedi: {}
  }
*/

state = store(state, newJediSelection);
console.log("Merged reducers", state);

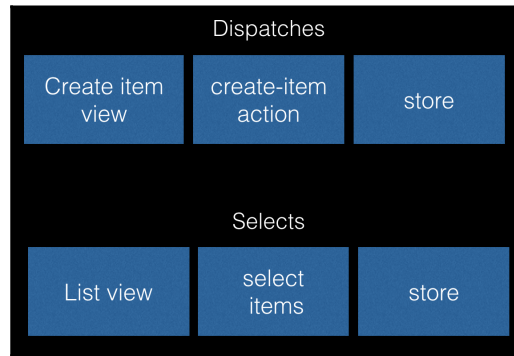
console.log(state);
/*
  {
    jedis: [{ name: 'Yoda' }, { name: 'Vader'}],
    selectedJedi: { name: 'Yoda' }
  }
*/

```

From what we can see here, our `store()` function does nothing more than return an object. The returned object is our current state. What we choose to call the properties of the state object is what we want to refer to when displaying the content of the store. If we want to change the state of our store, we need to invoke the `store()` function anew and provide it with an action that represents the intent of our change.

Data flow

OK, so we know about actions, reducers, and manipulating the state in a pure way. What about putting all this in practice in a real application? How would we do that? Let's try to model the data flow of our application. Imagine that we have a view that handles adding an item to a list and a view that handles showing the list. Then, our data flow could look like the following:



In the case of the create item view, we enter the data we need to create an item and then we dispatch an action, create-item, which ends up adding the item to the store. In our other data flow, we simply have a list view that selects the items from the store, which leads to the list view being populated. We realize that in a real application there may be the following steps:

1. User interaction
2. Creation of an action that represents our intent
3. Dispatching an action, which leads to our state changing its state

The preceding steps are true for our create item view. For our list view, we simply want to read from a store and display the data. Let's try to make this more tangible and turn at least the Redux part into actual code.

Creating the action

We will start by creating an action creator, a helper function that helps us create actions:

```
// dataflow/actions.js

export function createItem(title){
  return { type: "CREATE_ITEM", payload: { title: title } };
}
```

Creating a controller class – create-view.js

Now imagine that we are inside the code of the view that handles creating items; it may look something like this:

```
// dataflow/create-view.js

import { createItem } from "../actions";
import { dispatch, select } from "../redux";

console.log("create item view has loaded");

class CreateItemView {
  saveItem() {
    const elem = document.getElementById("input");
    dispatch(createItem(elem.value));
    const items = select("items");
    console.log(items);
  }
}

const button = document.getElementById("saveButton");
const createItemWiew = new CreateItemView();

button.addEventListener("click", createItemWiew.saveItem);

export default createItemWiew;
```

OK, so, in our `create-view.js` file we create a `CreateItemView` class that has a `saveItem()` method on it. The `saveItem()` method is the first responder to a button click on a button with the ID `saveButton`. When the button is clicked, our `saveItem()` method is invoked, which ends up calling our `dispatch` function with the `createItem()` action method, which in turn is using the input elements value as input, like so:

```
dispatch(createItem(elem.value));
```

Creating a store implementation

We have yet to create the `dispatch()` method, so we will do that next:

```
// dataflow/redux.js

export function dispatch(action) {
  // implement this
}
```

What we can see from the preceding code is that we have a `dispatch()` function, which is one of the things we export from this file. Let's try to fill in the implementation:

```
// dataflow/redux-step1.js

// 1)
function itemsReducer(state = [], action) {
  switch(action.type) {
    case "CREATE_ITEM":
      return [...state, Object.assign(action.payload)];
    default:
      return state;
  }
}

// 2)
let state = {
  items: []
};

// 3)
function store(state = { items: [] }, action) {
  return {
    items: itemsReducer(state.items, action)
  };
}
```

```
// 4)
export function getState() {
  return state;
}

// 5)
export function dispatch(action) {
  state = store(state, action);
}
```

Let's explain what we did there from the top. We first define a reducer 1) called `itemsReducer`, which can produce a new state given a new item. Thereafter, we create a state variable, which is our state 2). This is followed by the `store()` function 3), which is a function to set up which property goes together with which reducer. Thereafter, we define a function called `getState()` 4), which returns our current state. Lastly, we have our `dispatch()` function 5), which just invokes the `store()` function with the action we provide it.

Testing out our store

Now it is time to put our code to use; first, we will create a `redux-demo.js` file to test out our Redux implementation, then we will polish it a bit, and lastly we will use it in the view we created earlier:

```
// dataflow/redux-demo.js

import { dispatch, getState, select, subscribe } from "../redux";

const { addItem } = require("../actions");
subscribe(() => {
  console.log("store changed");
});
console.log("initial state", getState());
dispatch(addItem("A book"));
dispatch(addItem("A second book"));
console.log("after dispatch", getState());
console.log("items", select("items"));

/*
this will print the following

state before: { items: [] }
state after: { items: [{ title: 'a new book' }] }
*/
```

Cleaning up the implementation

OK, so our Redux implementation seems to be working. It's time to clean it up a bit. We need to move the reducer out into its own file, like so:

```
// dataflow/reducer.js

function itemsReducer(state = [], action) {
  switch(action.type) {
    case "CREATE_ITEM":
      return [...state, Object.assign(action.payload) ];
    default:
      return state;
  }
}
```

It's also a good idea to add a `select()` function to the store as we sometimes don't want to move a full state back, only part of it. Our list view will benefit from the use of the `select()` function. Let's add that next:

```
// dataflow/redux-stepII.js

// this now refers to the reducers.js file we broke out
import { itemsReducer } from "../reducers";

let state = {
  items: []
};

function store(state = { items: [] }, action) {
  return {
    items: itemsReducer(state.items, action)
  };
}

export function getState() {
  return state;
}

export function dispatch(action) {
  state = store(state, action);
}

export function select(slice) {
  return state[slice];
}
```

Creating our second controller class – list-view.js

Let's now shift our focus to our `list-view.js` file that we are yet to create:

```
// dataflow/list-view.js

import { createItem } from "../actions";
import { select, subscribe } from "../redux";

console.log("list item view has loaded");

class ListItemsView {
  constructor() {
    this.render();
    subscribe(this.render);
  }

  render() {
    const items = select("items");
    const elem = document.getElementById("list");

    elem.innerHTML = "";
    items.forEach(item => {
      const li = document.createElement("li");
      li.innerHTML = item.title;
      elem.appendChild(li);
    });
  }
}

const listItemsView = new ListItemsView();
export default listItemsView;
```

OK, so we utilize the `select()` method and get a slice of state from our state from the `redux.js` file we created. Thereafter, we render the response. As long as these views are on different pages, we will always get the latest version of `items` arrays from our state. However, if these views are visible at the same time, then we have a problem.

Adding subscription capability to our store

Somehow, the list view needs to listen to changes in the store so that it can rerender when a change happens. A way to do that is, of course, to set up some kind of listener that triggers an event when a change happens. If we, as a view, subscribe to such changes, then we can act accordingly and rerender our view. There are different ways of accomplishing this: we can either just implement an Observable pattern or use a library, such as `EventEmitter`. Let's update our `redux.js` file to make it so:

```
// dataflow/redux.js

import { itemsReducer } from "../reducer";
import EventEmitter from "events";
const emitter = new EventEmitter();

let state = {
  items: []
};

function store(state = { items: [] }, action) {
  return {
    items: itemsReducer(state.items, action)
  };
}

export function getState() {
  return state;
}

export function dispatch(action) {
  const oldState = state;
  state = store(state, action);
  emitter.emit("changed");
}

export function select(slice) {
  return state[slice];
}

export function subscribe(cb) {
  emitter.on("changed", cb);
}
```

Creating a program

So far, we have created a bunch of files, namely, the following:

- `redux.js`: Our store implementation.
- `create-view.js`: A controller that listens to inputs and button presses. The controller will read the input on a button press and dispatch the input's value so it is saved in the store.
- `list-view.js`: Our second controller, responsible for showing the content of the store.
- `todo-app.js`: The starter file that creates our entire application (we have yet to create this).
- `index.html`: The UI for our application (we are yet to create this).

Setting up our environment

Maybe you have noticed that we are using import statements used for ES6 modules? There are many ways to make that work, but we choose a modern option, namely, utilizing webpack. We will need to do the following to set up webpack successfully:

- Install the npm libraries `webpack` and `webpack-cli`
- Create a `webpack.config.js` file and specify the entry point of your application
- Add an entry to the `package.json` file so that we can build and run our app with a simple `npm start`
- Add a HTTP server so that we can show the app

We can install the needed libraries by typing:

```
npm install webpack webpack-cli --save-dev
```

Thereafter, we need to create our config file, `webpack.config.js`, like so:

```
// dataflow/webpack.config.js

module.exports = {
  entry: './todo-app.js',
  output: {
    filename: 'bundle.js'
  },
  watch: true
};
```

In the preceding code, we are stating that the entry point should be `todo-app.js` and also that the output file should be called `bundle.js`. We also ensure our bundle will be rebuilt by setting `watch` to `true`. Let's add the needed entry to `package.json` by adding the following to the `script` tag:

```
// dataflow/package.json excerpt

"scripts": {
  "start" : "webpack -d"
}
```

Here, we are defining a start command that invokes `webpack` with the flag `-d`, which means it will generate source maps, making for a nice debug experience.

For our last setup step, we need a HTTP server so that we can display our app. `Webpack` itself has one called, `webpack-dev-server`, or we could use `http-server`, which is an NPM package. This is a pretty simple application so either will do fine.

Creating the missing files and running our program

Our application needs a UI so let's create that:

```
// dataflow/dist/index.html

<html>
  <body>
    <div>
      <input type="text" id="input">
      <button id="saveButton">Save</button>
    </div>
    <div>
      <ul id="list"></ul>
    </div>
    <button id="saveButton">Save</button>
    <script src="bundle.js"></script>
  </body>
</html>
```

So, here we have an input element and a button that we can press to save a new item. This is followed by a list, where our content will be rendered.

Let's create the `todo-app.js` next. It should look like the following:

```
// dataflow/todo-app.js

// import create view
import createView from "../create-view";
// import list view
import listView from "../list-view";
```

Here, we are requiring in the two controllers so we can collect input as well as displaying store content. Let's try out our application by typing `npm start` in the Terminal window. This will create the `bundle.js` file in a `dist` folder. To display the app, we need to fire up another terminal window and place ourselves in the `dist` folder. Your `dist` folder should consist of the following files:

- `index.html`
- `bundle.js`

Now we are ready to launch the app by typing `http-server -p 5000`. You will be able to find your app at `http://localhost:5000` in your browser:



We see our expected application with an input element and a button, and we see our console to the right showing us that both our controllers were loaded. Additionally, we see the content of the `items` property of our store object, which points to an empty array. This is expected as we haven't added any items to it yet. Let's add an item to our store by adding a value to our input element and pressing the **Save** button:



On the right, we can see that our store now contains an item but our UI is not updated. The reason for that is that we don't actually subscribe to changes. We can change that by adding the following piece of code to our list-view.js controller file:

```
// dataflow/list-view.js

import { createItem } from "../actions";
import { select, subscribe } from "../redux";

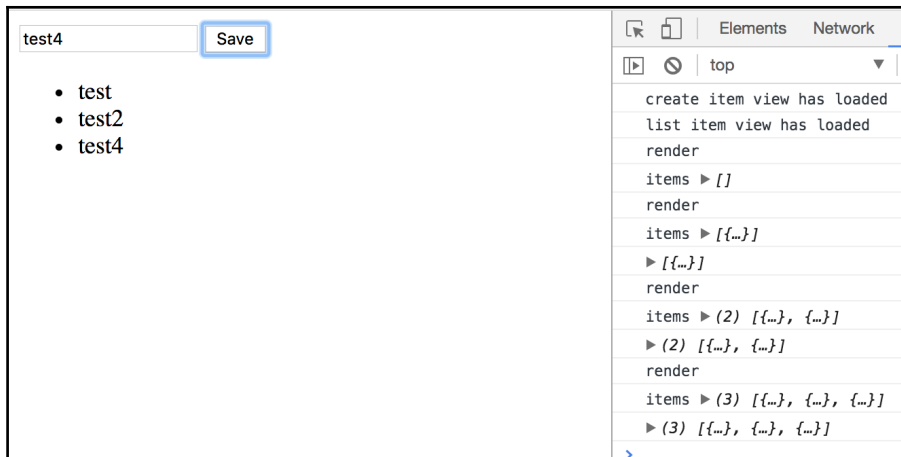
console.log("list item view has loaded");

class ListItemsView {
  constructor() {
    this.render();
    subscribe(this.render);
  }

  render() {
    const items = select("items");
    const elem = document.getElementById("list");
    elem.innerHTML = "";
    console.log("items", items);
    items.forEach(item => {
      const li = document.createElement("li");
      li.innerHTML = item.title;
      elem.appendChild(li);
    });
  }
}

const listItemsView = new ListItemsView();
export default listItemsView;
```

Now our app renders as it should and should look something like this, providing that you added a few items:



Dealing with asynchronous calls

Dispatching actions is always done synchronously. Data is fetched through AJAX asynchronously, so how do we get asynchronous to play well with Redux?

You should define your Redux states in the following way when setting up an asynchronous call:

- Loading: Here, we have the chance to show a spinner, not render part of the UI, or convey to the user in some other way that the UI is waiting for something
- Data successfully fetched: You should set a state for the fetched data
- Error happened: You should record the error somehow so that you are able to tell the user that an error occurred

You, by convention, use the word `fetch` to indicate that you are fetching data. Let's look at an example of what that might look like. First off, let's start by defining the steps we need to take:

1. Create a reducer. This should have the ability to set different states depending on whether we are waiting for the response, got the response, or an error happened.
2. Create actions. We need a file of actions that supports our states mentioned earlier; creating this is more about convenience.

3. Update our `redux.js` file to use our new reducer.
4. Test our creation out.

Let's say we are fetching a book from an API. We should have a reducer that looks like the following:

```
// async/book-reducer.js

let initialState = {
  loading: false,
  data: void 0,
  error: void 0
};

const bookReducer = (state = initialState, action) => {
  switch(action.type) {
    case 'FETCH_BOOK_LOADING':
      return {...state, loading: true };
    case 'FETCH_BOOK_SUCCESS':
      return {...state, data: action.payload.map(book => ({ ... book })) };
    case 'FETCH_BOOK_ERROR':
      return {...state, error: { ...action.payload }, loading: false };
  }
}

module.exports = bookReducer;
```

Now that we have covered the reducer bit, let's move on to the creation of actions. It will look like the following:

```
// async/book-actions.js

const fetchBookLoading = () => ({ type: 'FETCH_BOOK_LOADING' });
const fetchBookSuccess = (data) => ({ type: 'FETCH_BOOK_SUCCESS', payload:
data });
const fetchBookError = (error) => ({ type: 'FETCH_BOOK_ERROR', payload:
error });

module.exports = {
  fetchBookLoading,
  fetchBookSuccess,
  fetchBookError
};
```

Now we need to turn to our store file and update it:

```
// async/redux.js

const bookReducer = require('./book-reducer');
const EventEmitter = require('events');
const emitter = new EventEmitter();

let state = {
  book: {}
};

function store(state = {}, action) {
  return {
    book: bookReducer(state.book, action)
  };
}

function getState() {
  return state;
}

function dispatch(action) {
  const oldState = state;
  state = store(state, action);
  emitter.emit('changed');
}

function select(slice) {
  return state[slice];
}

function subscribe(cb) {
  emitter.on('changed', cb);
}

module.exports = {
  getState, dispatch, select, subscribe
}
```

Creating a demo with Redux and asynchronous

Now the time has come to test everything out. What we are interested in here is ensuring that our store state behaves the way we want it to. We want the store to reflect the fact that we are loading the data, receiving the data, and if there is an error, that should be reflected as well. Let's start out by mimicking an AJAX call:

```
const { fetchBookLoading, fetchBookSuccess, fetchBookError } =
require('./book-actions');
const { dispatch, getState } = require('./redux');

function fetchBook() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve({ title: 'A new hope - the book' });
    }, 1000);
  })
}
```

As our next order of business, let's set up some logging for the state and dispatch our first action, `fetchBookLoading`, which indicates that an AJAX request is underway. This is where, ideally, we would want to reflect this state in the UI and show a spinner or similar:

```
console.log(getState());
// { book: {} }

dispatch(fetchBookLoading());

console.log(getState());
// { book: { loading: true } }
```

The last step is about a call to our `fetchBook()` method and setting the store state appropriately:

```
async function main() {
  try {
    const book = await fetchBook();
    dispatch(fetchBookSuccess(book));
    console.log(getState());
    // { book: { data: { title: 'A new hope - the book'}, loading: false } }
  } catch(err) {
    dispatch(fetchBookError(err));
    console.log(getState());
  }
}
```

```

    // { book: { data: undefined, error: { title: 'some error message' } } }
  }
}

main();

```

We have so far described our demo in pieces from top to bottom. The full code should read like this:

```

// async/demo.js

const { fetchBookLoading, fetchBookSuccess, fetchBookError } =
require('./book-actions');
const { dispatch, getState } = require('./redux');

function fetchBook() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve({ title: 'A new hope - the book' });
    }, 1000);
  })
}

console.log(getState());
dispatch(fetchBookLoading());
console.log(getState());

async function main() {
  try {
    const book = await fetchBook();
    dispatch(fetchBookSuccess(book));
    console.log(getState());
  } catch(err) {
    dispatch(fetchBookError(err));
    console.log(getState());
  }
}

main();

```

As you can see, there is really not much to dealing with asynchronous, you just need to dispatch the suitable state once the asynchronous action has run its course. There are libraries for dealing with asynchronous though. If you are a React user it might be worth looking into Sagas and if Angular is your flavor, then NgRx and effects is your go-to. The reason for separate libraries existing for this is to say that asynchronous interactions, especially AJAX ones, are considered side-effects and as such they are outside the *normal* flow. Ultimately, it's up to you if you feel you need such a library.